

CPT102 Data Structures:

Tutorial 4: Merge sort

Thomas Selig

XJTLU

April 10, 2026

ICT announcements: schedule and questions

- Date and time are now decided.
 - Date: **Wednesday, 15 April, at 19:00**
 - Location: in one of five labs, check Learning Mall spreadsheet!!
- Duration: 45 minutes
- Question format:
 - 3 MCQs: one correct answer from four options
 - 3 Short-answer questions: answer = a single word, a number, a Big-Oh estimate, a symbol, a list, etc.
 - 2 (Java) coding questions, fill-in-the-blank

Question types mimic those of final exam.

- Mock-quiz now online: fewer questions, but same format.

ICT: what can you bring?

ICT will run using *Safe-Exam Browser* (SEB) software, same as CPT111 (that you all took last semester).

- **Allowed materials:** any PDF or .java files
 - Bring with you on USB, or put on BOX and download before entering SEB.
- **Allowed software:** PDF reader (Adobe), plain text file reader (notepad).
 - No IDEs or Word/Excel!
- **Allowed URLs:** only the Quiz homepage.
 - No other pages will be accessible, including LM course page(s)! So make sure to download any materials for CPT102 separately.

Contents of today's tutorial

The main goal: **implementing Merge Sort**

- Overview of merge sort
- A deep dive into MERGE (the key step)
- Implementation in Java

Merge sort: overview

The idea behind merge sort is simple.

- ➊ **Divide:** we break the array into two sub-arrays of (roughly) equal length.
- ➋ **Conquer:** recursively sort each sub-array.
- ➌ **Combine:** merge the two sorted halves into a single sorted array.

Example: input array is [5, 1, 2, 6, 4, 3].

- ➊ **Divide:** sub-arrays are [5, 1, 2] (left) and [6, 4, 3] (right).
- ➋ **Conquer:** recursively sort sub-arrays to get [1, 2, 5] (left) and [3, 4, 6] (right).
- ➌ **Combine:** merge the two sorted halves to get [1, 2, 3, 4, 5, 6].

Merge sort: partial implementation (1/2)

Algorithm 1: merge sort

Procedure MERGESORT($T[]$ arr)

temp $\leftarrow$ new $T[\text{arr.length}]$;

MERGESORTHELPER(arr, temp, 0, arr.length);

Procedure MERGESORTHELPER(arr, temp, start, end)

if $\text{end} - \text{start} \leq 1$ **then**

return ; ▷ If the sub-array has length at most 1, nothing to do

mid $\leftarrow$ (start + end) / 2;

MERGESORTHELPER(arr, temp, start, mid);

MERGESORTHELPER(arr, temp, mid, end);

MERGE(arr, temp, start, mid, end);

- As usual, helpers with additional index parameters.
- Here, an additional array temp is created, shared across helpers and recursive calls, to optimise memory usage.

Merge sort: partial implementation (2/2)

- Pseudocode on previous slide is obviously incomplete: need to implement MERGE procedure.
- Still useful: we've *decomposed* the problem, and *reduced* it to the MERGE problem.
 - The principle of *decomposition* is a very important problem-solving tool in Computer Science.
 - It means breaking down a problem into smaller pieces, solving each smaller piece in turn, then using this to solve the original problem.
 - One of the four so-called “pillars” of *computational thinking*.

The MERGE problem

Given an array `arr` and three indices $\text{start} \leq \text{mid} \leq \text{end}$, such that `arr[start:mid]` and `arr[mid:end]` are sorted, modify `arr` so that `arr[start:end]` becomes sorted.

The merge operation: implementation

Algorithm 2: merging two sorted arrays

Input: An array `arr` and indices `start`, `mid`, `end` such that `arr[start:mid]` and `arr[mid:end]` are sorted

Output: The modified array `arr` such that `arr[start:end]` is sorted

`COPY(arr, temp, start, end);` ▷ copy the sub-array `arr[start:end]` into `temp`

`i ← start, j ← mid, k ← start;`

▷ indices for left sub-array, right sub-array, merged result

while `i < mid` **and** `j < end` **do**

if `LESS(temp[j], temp[i])` **then**

`arr[k] ← temp[j], j ← j + 1;`

else

`arr[k] ← temp[i], i ← i + 1;`

`k ← k + 1;`

while `i < mid` **do**

`arr[k] ← temp[i], i ← i + 1, k ← k + 1;`

 ▷ copy remaining elements from left-half, if any

▷ no need to copy remaining elements from right-half, as already in correct place

Example: MERGE

Input array: $[4, 6, 10, 12^1 \mid 2, 8, 12^2, 14]$.

- The idea: use i to track the left sub-array, j to track the right sub-array, and k to track the merged result.
- Start by copying the array into temp (i and j will track on temp, k will track on arr).
- Initially: $i = k = 0$, $j = \text{mid} = 4$

arr:

4	6	10	12 ¹	2	8	12 ²	14
---	---	----	-----------------	---	---	-----------------	----

temp:

4	6	10	12 ¹	2	8	12 ²	14
---	---	----	-----------------	---	---	-----------------	----

Intuitively, we think of arr as being “empty” initially: we fill it with the correct values while iterating over the left and right sub-arrays of temp **simultaneously**.

- At each step, compare $\text{temp}[i]$ and $\text{temp}[j]$: assign the smallest of the two values to $\text{arr}[k]$.
 - If equal, choose the left one to preserve stability.

Example: MERGE

- Start with $i = k = 0$, $j = \text{mid} = 4$

arr:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

temp:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

- Compare $\text{temp}[i] = 4$ and $\text{temp}[j] = 2$, assign smallest value 2 to $\text{arr}[k]$, and increment j and k .

arr:

2	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

temp:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

- Next compare $\text{temp}[i] = 4$ and $\text{temp}[j] = 8$, assign smallest value 4 to $\text{arr}[k]$, and increment i and k .

arr:

2	4	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

temp:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

Example: MERGE

- We have $6 < 8$, so $\text{arr}[k] \leftarrow 6$, increment i and k .

arr:

2	4	6	12^1	2	8	12^2	14
---	---	---	--------	---	---	--------	----

temp:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

- We have $8 < 10$, so $\text{arr}[k] \leftarrow 8$, increment j and k .

arr:

2	4	6	8	2	8	12^2	14
---	---	---	---	---	---	--------	----

temp:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

- We have $10 < 12^2$, so $\text{arr}[k] \leftarrow 10$, increment i and k .

arr:

2	4	6	8	10	8	12^2	14
---	---	---	---	----	---	--------	----

temp:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

- We have $12^1 \leq 12^2$, so $\text{arr}[k] \leftarrow 12^1$, increment i and k .

arr:

2	4	6	8	10	12^1	12^2	14
---	---	---	---	----	--------	--------	----

temp:

4	6	10	12^1	2	8	12^2	14
---	---	----	--------	---	---	--------	----

Example: MERGE

- At this point, we have reached the end of the left sub-array with our i pointer.
- Observations:
 - We have $\text{arr}[k:\text{end}] = \text{temp}[k:\text{end}]$, already sorted!
 - By construction, $\text{arr}[\text{start}:k]$ is sorted.
 - Also, by construction (last step), we have $\text{arr}[k-1] \leq \text{arr}[k]$
- Therefore, the full array arr is sorted!
 - Some implementations copy $\text{temp}[k:\text{end}]$ back into arr , but it's not necessary.
 - HOWEVER! If we reach the end of the right sub-array first (for example, replace 12^1 with 16 in left sub-array), we would need to copy remaining elements of left sub-array at the end of arr .

Coding time!

On Learning Mall, you will find a skeletal `Merge.java` class, containing all method headers.

- This week, for simplicity, we are dealing directly with an array of integers.
- Start by implementing the `merge()` method, based on the pseudocode implementation covered previously.
- Then, use this to implement the `sortHelper()` and finally `sort()` method.
- Test this in the `main()` method: create an array of 10 random integers between 0 and 100 (use a for loop, and the `java.util.Random` class), print it, then sort it and print the sorted version.

Happy coding!

Lab Exercise 1: Modifying input types

- Currently our `merge.sort()` method requires an array of integers to function.
- What about an array of doubles? Write a new method `sort(double[] arr)` that sorts an input array of double values.
 - You may also add new helper methods as appropriate.
 - Why can we have two methods in the same class with the same name?
- Now, using work from last week, write a generic method `sort(T[] arr)` to be able to sort arrays of any type which implements the `Comparable` interface.
- Test your new methods in `main()`, for example by re-using the `Student` class from last week's Lab, or sorting an array of 10 random double values between 0 and 1.

Lab Exercise 2: Counting MERGE operations

- Through judicious use of a static class counter variable, modify the methods in `Merge.java` (for example, the `int[]` sorting method and its helpers) so that the `sort()` method sorts the array and returns the number of merge operations required.
 - For this, introduce a static class variable `counter`, initialise it to 0 at the start of the `sort()` method, and ensure it is incremented in `merge()`.
- Now test this for `n` (the length of the input array) ranging from 1 to a large number in multiples of 2. How does the number of merge operations evolve when `n` is multiplied by 2 at each step. Why?
 - Check the lecture notes to see how many times `merge()` would typically be called for large `n`.